



AULA 04

Funções e Procedimentos

Dividir o programa em pedaços · parâmetros por valor e por referência

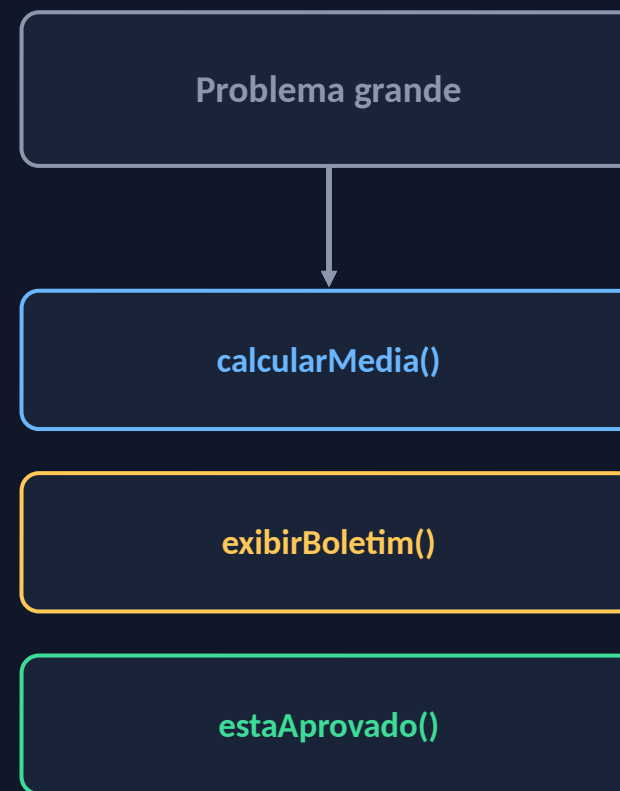
```
$ int soma(int a, int b) { return a + b; }  
> escreva uma vez, use quantas quiser
```



Por que usar funções?

Uma função é um bloco de código com nome, que resolve uma tarefa específica e pode ser chamado várias vezes.

- Reaproveitar: escreve uma vez, usa quantas vezes quiser.
- Organizar: cada função faz uma coisa só.
- Dividir para conquistar: um problema grande vira vários pequenos.
- Facilitar a manutenção: corrige num lugar só.



cada pedaço vira uma função

>_

Anatomia de uma função em C

Toda função tem quatro partes. Observe cada cor:

● ● ● soma.c

```
int soma(int a, int b) {  
    return a + b;  
}
```

chamada: `int r = soma(3, 4); // r = 7`

Tipo de retorno

o tipo do valor devolvido (int, float, char). void = não devolve nada.

Nome

como chamamos a função no programa.

Parâmetros

as entradas que a função recebe para trabalhar.

return

devolve o resultado para quem chamou a função.

>_ Função x Procedimento

A diferença é simples: a função DEVOLVE um valor; o procedimento apenas EXECUTA uma ação.

FUNÇÃO — devolve um valor

```
funcao.c

int soma(int a, int b) {
    return a + b;
}
```

- Tem um tipo de retorno (int, float...).
- Usa return para devolver o resultado.
- Serve para calcular algo: soma, média, area.

PROCEDIMENTO — só executa (void)

```
procedimento.c

void saudacao() {
    printf("Ola, mundo!\n");
}
```

- O tipo de retorno é void (nada).
- Não tem return de valor.
- Serve para fazer uma ação: imprimir, desenhar.

Em C: os dois são “funções”. Chamamos de procedimento quando o retorno é void.

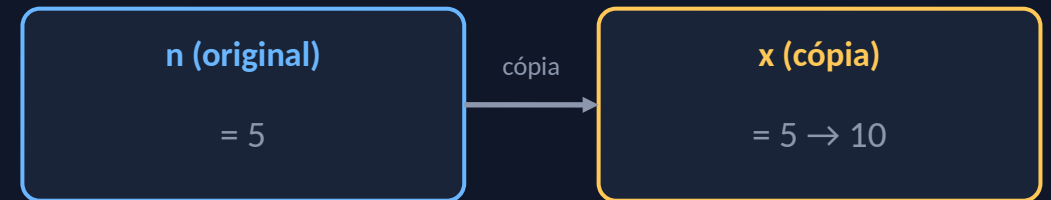
>_ Passagem por VALOR: a função recebe uma cópia

Por valor, a função trabalha com uma CÓPIA do dado. Mexer na cópia não muda o original.

por_valor.c

```
void dobrar(int x) {  
    x = x * 2;    // muda só a cópia  
}  
  
int main() {  
    int n = 5;  
    dobrar(n);  
    printf("%d", n); // 5 (não mudou!)  
}
```

O que acontece na memória



A função dobra o x (a cópia). O n lá do main continua 5.

Saída: 5

>_ Passagem por REFERÊNCIA: a função recebe o endereço

Por referência, passamos o ENDEREÇO da variável (com &). Aí a função mexe direto no original.

por_referencia.c

```
void dobrar(int *x) {  
    *x = *x * 2; // muda o original  
}  
  
int main() {  
    int n = 5;  
    dobrar(&n);    // passa o endereço  
    printf("%d", n); // 10 (mudou!)  
}
```

O que acontece na memória



Lembram do scanf? scanf("%d", &n) usa o MESMO & — passando o endereço para a função escrever no seu n.

Saída: 10

>_

O caso clássico: trocar dois números

Trocar o valor de x e y é o exemplo que mostra tudo: por valor NÃO funciona; por referência funciona.

POR VALOR — não troca

● ● ● valor.c

```
void trocar(int a, int b) {  
    int t = a; a = b; b = t;  
}  
// x=1, y=2 → trocar(x, y)  
// resultado: x=1, y=2 (igual)
```

Só as cópias foram trocadas — o x e o y do main nem sentiram.

POR REFERÊNCIA — troca!

● ● ● referencia.c

```
void trocar(int *a, int *b) {  
    int t = *a; *a = *b; *b = t;  
}  
// x=1, y=2 → trocar(&x, &y)  
// resultado: x=2, y=1 (trocou)
```

Passando os endereços, a função mexe direto no x e no y originais.

Regra de bolso: a função precisa MUDAR a variável de quem chamou? Use referência (&/*). Só precisa LER o valor? Use por valor.

>_

Exercícios para fazer em sala

Vamos construir juntos: escreva a função/procedimento, chame no main e faça o teste de mesa.

1

Função soma

Crie uma função que recebe dois inteiros e retorna a soma.

2

Procedimento saudação

Crie um procedimento (void) que recebe um nome e imprime "Olá, <nome>!".

3

Por valor: dobrar

Faça void dobrar(int x) e mostre que o valor no main NÃO muda.

4

Por referência: dobrar

Refaça com ponteiro para que o valor no main MUDE.

5

Desafio: trocar

Escreva trocar(int *a, int *b) que troca os valores de duas variáveis.

>_ Gabarito

● ● ● ex1.c

```
// 1) Função soma
int soma(int a, int b) {
    return a + b;
}
```

● ● ● ex2.c

```
// 2) Procedimento saudação
void saudacao(char nome[]) {
    printf("Ola, %s!\n", nome);
}
```

● ● ● ex3.c

```
// 3) Por valor (n continua igual)
void dobrar(int x) {
    x = x * 2;
}
```

● ● ● ex4.c

```
// 4) Por referência (n muda)
void dobrar(int *x) {
    *x = *x * 2;
} // chamada: dobrar(&n);
```

● ● ● ex5.c

```
// 5) Trocar (por referência)
void trocar(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
} // chamada: trocar(&x, &y);
```

Dica de teste de mesa: no ex.3, n fica 5. No ex.4 e 5, os valores mudam porque a função recebeu o endereço.



Recapitulando

- Função: bloco com nome que resolve uma tarefa e pode ser reusado.
- Função DEVOLVE um valor (tem return); procedimento é void (só executa).
- Parâmetros são as entradas; argumentos são os valores passados.
- Por valor: a função recebe uma CÓPIA — o original não muda.
- Por referência: passamos o ENDEREÇO (&) — a função muda o original.
- O & do scanf sempre foi passagem por referência!

Guia rápido

- Precisa de um valor de volta? → função
- Só executar uma ação? → procedimento (void)
- Precisa mudar a variável de fora? → referência (&, *)
- Só ler o valor? → por valor

Escreva uma vez, use sempre — e mexa no original só quando precisar.